Trabajo Práctico 6 – Ejercicios extra

SISTEMAS OPERATIVOS

Tracks

1. Permisos, bit SUID (Set User ID) y sticky bit (5/10) **3**
1.1. Ejecutable con bit SUID 4

2. Ⓢ Sistemas de archivos remotos (8/10) **5**

3. Ⓢ Dispositivos de bloque remotos (8/10) **6**

4. LVM: volúmenes lógicos (4/10) **7**

5. Ⓢ Leyendo MBR (6/10) **8**

6. Journaling? qceso? (6/10) **10**

7. Cuotas de disco (2/10) **11**

8. Sockets de dominio y transferencia (6/10) **12**

9. Ⓢ Overlays (7/10) **13**

10. Ⓢ Boss: Dispositivo de caracteres (10/10) **14**

11. RAID en software (9/10) **16**

(Ⓢ= hay parte por puntos)

1. Permisos, bit SUID (Set User ID) y sticky bit (5/10)

TRACK – 5/10

Descripción: entender cómo los permisos de búsqueda y escritura de un directorio controlan el acceso a sus nombres, y observar el efecto del sticky bit y del bit SUID en un ejecutable.

INFO

Creá dos cuentas de prueba y asignales una contraseña:

```
$ sudo useradd -m tp6-a
$ sudo passwd tp6-a
$ sudo useradd -m tp6-b
$ sudo passwd tp6-b
```

Al terminar, salí de ambas cuentas y eliminalas:

```
$ sudo userdel -r tp6-a
$ sudo userdel -r tp6-b
```

Trabajá en un directorio compartido por dos cuentas. Creá archivos con cada cuenta y probá combinaciones de permisos usando `chmod`, `umask` y `ls -ld`. Observá por separado:

- qué permite el permiso de lectura sobre un directorio;
- qué permite el permiso de escritura;
- por qué el permiso de búsqueda permite recorrer un nombre conocido;
- quién puede eliminar o renombrar un archivo en un directorio compartido.

Activá el sticky bit con `chmod +t` y repetí las operaciones desde ambas cuentas. Relacioná el resultado con el directorio `/tmp` y con sus permisos.

1.1. Ejecutable con bit SUID

En un ejecutable, el bit SUID hace que el identificador efectivo de usuario (effective user ID) del proceso pase a ser el propietario del archivo. El identificador real sigue correspondiendo a quien lo ejecutó. El programa siguiente imprime ambos identificadores:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main(void) {
    printf("UID real: %lu\n", (unsigned long)getuid());
    printf("UID efectivo: %lu\n", (unsigned long)geteuid());
    return 0;
}
```

Desde la raíz del repositorio, compilá y ejecutá el programa:

```
$ cc -Wall -Wextra -o whoami examples/tp6/whoami.c
$ ./whoami
```

Consultá las opciones del montaje que contiene `/usr/local/bin`; el núcleo ignora el bit SUID si ese montaje usa `nosuid`. Instalá el ejecutable como propiedad de `root` y ejecutalo como la segunda cuenta:

```
$ findmnt -T /usr/local/bin -o TARGET,OPTIONS
$ sudo install -o root -g root -m 0755 whoami /usr/local/bin/tp6-
    whoami
$ sudo -u tp6-b /usr/local/bin/tp6-whoami
```

Activá el bit SUID, inspeccioná los permisos y repetí la ejecución. Relacioná los identificadores impresos con `id -u tp6-b`, `id -u root` y la `s` que aparece en `ls -l`:

```
$ sudo chmod u+s /usr/local/bin/tp6-whoami
$ ls -l /usr/local/bin/tp6-whoami
$ sudo -u tp6-b /usr/local/bin/tp6-whoami
$ sudo chmod u-s /usr/local/bin/tp6-whoami
$ sudo rm /usr/local/bin/tp6-whoami
$ rm whoami
```

2. Ⓢ Sistemas de archivos remotos (8/10)

TRACK – 8/10

Descripción: montar un directorio remoto con NFS (Network File System) y con SSH (Secure Shell), usando SSHFS (SSH File System), y comparar cómo se integra cada uno al árbol local.

Usá dos máquinas Linux conectadas a una red privada. En una, exportá un directorio por NFS; en la otra, montalo y consultá el resultado con `findmnt`, `stat` y operaciones de lectura y escritura. Después montá un directorio de la primera máquina con SSHFS.

También podés montar recursos NFS y SSHFS desde Windows: NFS requiere habilitar el cliente correspondiente, y SSHFS requiere instalar WinFsp y SSHFS-Win. La configuración puede llevar más pasos que en Linux.

Compará el punto de montaje, el tipo de sistema de archivos, los permisos observados, el efecto de perder la conexión y los procesos que intervienen. En la máquina servidor, consultá los archivos abiertos y los mensajes del servicio. En la máquina cliente, observá las operaciones de montaje y desmontaje.

BONUS – POR PUNTOS

Implementá un sistema de archivos FUSE (Filesystem in Userspace) que presente un archivo TAR (Tape Archive) mediante la interfaz sugerida:

```
tarview ARCHIVO.tar PUNTO_DE_MONTAJE
```

El programa debe exponer los directorios y archivos de un archivo TAR como un árbol de sólo lectura, sin extraer su contenido. Permití listar directorios, consultar metadatos y leer archivos. Desmontá el sistema al terminar.
Para solicitar los puntos, entregá una explicación breve con capturas de las pruebas. Prepará una demostración de la implementación.

3. $ Dispositivos de bloque remotos (8/10)

TRACK – 8/10

Descripción: presentar un dispositivo de bloques a una máquina remota mediante un protocolo de la lista.

En una red privada, prepará un archivo de imagen con un sistema de archivos ext4. Montalo localmente con `mount -o loop` en la máquina que hará de destino, escribí un archivo de prueba y ejecutá `sync`. Desmontalo antes de presentarlo como dispositivo de bloques. Elegí una tecnología disponible en el entorno e instalá y configurá el software de destino e iniciador:

- iSCSI (Internet Small Computer Systems Interface);
- NVMe-oF (Non-Volatile Memory Express over Fabrics) sobre TCP (Transmission Control Protocol);
- AoE (ATA (Advanced Technology Attachment) over Ethernet).

Configurá el destino respaldado por la imagen y conectá otra máquina como iniciador. Identificá allí el nuevo dispositivo de bloques, montá el sistema de archivos y comprobá que aparece el archivo de prueba. Consultá el transporte y los mensajes del núcleo, y explicá qué funciones cumple cada extremo.

BONUS – POR PUNTOS

Repetí la configuración entre dos máquinas físicas distintas conectadas a la red del laboratorio. Presentá la topología, el protocolo elegido, las operaciones de lectura y escritura observadas y la limpieza del destino al finalizar.
Para solicitar los puntos, entregá una explicación breve con capturas y prepará una demostración del acceso entre las dos máquinas.

4. LVM: volúmenes lógicos (4/10)

TRACK – 4/10

Descripción: combinar dispositivos de bloques en volúmenes lógicos con LVM (Logical Volume Manager).

Creá y conectá dos discos virtuales vacíos en formato VHDX (Virtual Hard Disk v2). Identificalos con `lsblk` y consultá `pvs`, `vgs` y `lvs` antes de inicializarlos.

Convertí ambos discos en volúmenes físicos, agrupálos en un grupo de volúmenes y creá un volumen lógico. Creale un sistema de archivos ext4 (Fourth Extended Filesystem), montalo en un directorio y verificá su capacidad con `df`. Extendé el volumen lógico y el sistema de archivos, y observá cómo cambian los resultados de `lvs`, `lsblk` y `df`.

Usá `pvs`, `vgs` y `lvs` para relacionar los dispositivos físicos, el grupo y el volumen lógico. Observá cómo repartió LVM las extensiones entre los dos discos.

5. Ⓢ Leyendo MBR (6/10)

TRACK – 6/10

Descripción: implementar un lector del caso básico de una tabla de particiones MBR (Master Boot Record) y comprobarlo con un disco de prueba.

Creá una imagen de disco de 128 mebibytes (MiB) y conectala a un dispositivo de bucle (loop device) que detecte particiones:

```
$ truncate -s 128M disco.img
$ sudo losetup --find --show --partscan disco.img
```

Usá el dispositivo que informó `losetup` en `fdisk`. Creá una tabla DOS (Disk Operating System) con una partición primaria:

```
$ sudo fdisk DISPOSITIVO_LOOP
```

Dentro de `fdisk`, usá `o` para crear la tabla, `n` para agregar una partición con los valores predeterminados y `w` para escribir los cambios. Identificá la partición con `lsblk`, creale un sistema de archivos ext4, montala, escribí un archivo de prueba y desmontala. Después liberá el dispositivo de bucle:

```
$ lsblk -f DISPOSITIVO_LOOP
$ sudo mkfs.ext4 PARTICION
$ sudo mkdir -p /mnt/tp6-mbr
$ sudo mount PARTICION /mnt/tp6-mbr
$ printf 'prueba MBR\n' | sudo tee /mnt/tp6-mbr/prueba.txt
$ sync
$ sudo umount /mnt/tp6-mbr
$ sudo losetup -d DISPOSITIVO_LOOP
```

Implementá en C, C++ o Zig un programa que reciba la ruta de la imagen. Debe leer los primeros 512 bytes e imprimir la tabla de particiones. El MBR guarda cuatro entradas de 16 bytes a partir del byte 446 y la firma `0x55AA` en los bytes 510 y 511. Verificá la firma. Para cada entrada no vacía, mostrá el tipo, el sector inicial y la cantidad de sectores. Decodificá los campos de varios bytes con el orden little-endian (least significant byte first, byte menos significativo primero).

Con el dispositivo liberado y la partición desmontada, compilá y ejecutá el programa:

```
$ PROGRAMA disco.img
```

Compará su salida con `fdisk -l disco.img` y con los primeros 512 bytes que muestran `hexdump -C -n 512 disco.img`.

BONUS – POR PUNTOS

Extendé el programa para seguir una partición extendida y leer sus EBR (Extended Boot Record), mostrando las particiones lógicas que encuentre. Probalo con una imagen que tenga una partición extendida y al menos dos particiones lógicas, y compará la salida con `fdisk -l`.
Para solicitar los puntos, entregá una explicación breve con capturas de las pruebas y prepará una demostración del programa.

6. Journaling? qceso? (6/10)

TRACK – 6/10

Descripción: examinar la bitácora (journal) de un sistema ext4 (Fourth Extended Filesystem) y relacionar sus transacciones con cambios de metadatos.

Creá una imagen ext4 de práctica y consultá el superbloque con `dumpe2fs -h`. Identificá el nodo-i de la bitácora. Usá `debugfs` y su operación `logdump` sobre la imagen desmontada para explorar las transacciones que conserve.

Realizá cambios pequeños en la estructura de directorios y archivos, y compará lo que muestran las herramientas antes y después de desmontar el sistema. Relacioná los bloques de descriptor y de datos de la bitácora con la documentación del formato ext4.

La bitácora es circular y sus registros pueden reutilizarse o quedar vacíos después de sincronizar y desmontar el sistema. Considerá ese estado al interpretar los resultados y explicá qué transacciones muestra la imagen.

7. Cuotas de disco (2/10)

TRACK – 2/10

Descripción: limitar el espacio o la cantidad de nodos-i que puede usar una cuenta en un sistema de archivos.

En un sistema de archivos dedicado a este track, habilitá cuotas de usuario o de grupo. Consultá las herramientas `quota`, `edquota`, `quotaon` y `repquota`.

Configurá un límite blando y uno rígido para un usuario de prueba. Creá archivos hasta observar el aviso asociado al límite blando y luego el rechazo al superar el límite rígido. Repetí el experimento con un límite de nodos-i y compará el uso de espacio con la cantidad de archivos.

8. Sockets de dominio y transferencia (6/10)

TRACK – 6/10

Descripción: transferir archivos entre procesos locales con sockets de dominio Unix (Unix domain sockets) y comparar dos formas de mover datos.

Implementá un cliente y un servidor que se comuniquen mediante la familia `AF_UNIX` (Address Family Unix). El cliente solicita un archivo y el servidor lo envía por el socket. Definí cómo se informa el tamaño o el fin de la transferencia y cómo termina el cliente cuando el servidor cierra la conexión.

Primero transferí los datos con un ciclo de lectura y escritura. Después repetí la transferencia con la llamada al sistema (system call) `sendfile`, que mueve los datos desde el archivo al socket dentro del núcleo. Así, el proceso no copia cada bloque mediante un búfer propio. Medí el tiempo de transferencia y el uso de la unidad central de procesamiento (Central Processing Unit, CPU) con archivos grandes, repetí las corridas en condiciones equivalentes y compará las llamadas observadas con `strace`. La caché del sistema de archivos, el tamaño del archivo y el equipo influyen en los resultados; describí las condiciones de medición y explicá qué ventaja observaste. La documentación de `sendfile(2)` describe esta transferencia entre descriptores.

9. Ⓢ Overlays (7/10)

BONUS – POR PUNTOS

Configurá tu propio sistema de archivos OverlayFS (Overlay Filesystem) con directorios `lower`, `upper`, `work` y `merged`. Los directorios `upper` y `work` deben estar en el mismo sistema de archivos. Montá las capas, modificá un archivo que sólo exista en `lower` y observá cuándo aparece su copia en `upper`. Registrá también cómo representa OverlayFS la eliminación de una entrada.
Para solicitar los puntos, entregá una explicación breve con capturas de las capas antes y después de las operaciones.

10. Ⓢ Boss: Dispositivo de caracteres (10/10)

BONUS – POR PUNTOS

Descripción: relacionar los nodos de dispositivo de `/dev` con un módulo del núcleo que controle los indicadores luminosos de un teclado.
Las luces de Bloq Mayús (Caps Lock), Bloq Num (Num Lock) y Bloq Despl (Scroll Lock) son diodos emisores de luz (light-emitting diodes, LEDs). El subsistema de entrada (input subsystem) de Linux las controla con eventos de salida `EV_LED`; los códigos `LED_CAPSL`, `LED_NUML` y `LED_SCROLLL` corresponden a esos indicadores.
Usá `evtest` para identificar un teclado conectado que anuncie al menos `LED_CAPSL` y `LED_NUML`, y consultá su nodo `/dev/input/eventX`.
El núcleo debe aceptar la carga del módulo. En el entorno de práctica, desactivá el arranque seguro (Secure Boot) o firmá el módulo con una clave que el núcleo reconozca como confiable.
Implementá un módulo que registre un dispositivo de caracteres llamado `tp6-leds`. Conectalo al teclado mediante un manejador del subsistema de entrada (input handler) y abrí el dispositivo de entrada desde el manejador. En la operación `write`, usá `input_inject_event()` con la estructura de conexión `input_handle` asociada al teclado para enviarle eventos `EV_LED`. La operación debe interpretar un byte por escritura:

- el byte `1` enciende Bloq Mayús y deja sin cambios los demás indicadores;
- el byte `2` enciende Bloq Num y deja sin cambios los demás indicadores;
- el byte `0` apaga todos los indicadores asociados a eventos `EV_LED` que admita el teclado.

Instalá un compilador de C, `make` y los encabezados de desarrollo del núcleo cuya versión informa `uname -r`. En Debian o Ubuntu, podés instalarlos con:

```
$ sudo apt install build-essential linux-headers-$(uname -r)
```

Guardá el módulo en `tp6_leds.c`. En un archivo `Makefile` del mismo directorio, indicá el objeto que debe compilar Kbuild (Kernel Build System), el sistema de compilación del núcleo:

```
obj-m := tp6_leds.o
```

Esa regla produce `tp6_leds.ko`. Compilá contra los encabezados del núcleo en ejecución desde ese directorio:

```
$ make -C "/lib/modules/$(uname -r)/build" M="$PWD" modules
```

La documentación de Kbuild para módulos externos explica este formato. `insmod` carga el archivo `.ko` indicado; `lsmod` muestra los módulos

cargados y `dmesg` permite consultar los mensajes del núcleo. Probá la carga y consultá esas salidas:

```
$ sudo insmod ./tp6_leds.ko
$ lsmod | grep tp6_leds
$ sudo dmesg | tail -n 20
```

Probá la interfaz y observá los indicadores del teclado:

```
$ printf '1' | sudo tee /dev/tp6-leds
$ printf '2' | sudo tee /dev/tp6-leds
$ printf '0' | sudo tee /dev/tp6-leds
```

Definí el resultado de escribir otros valores o más de un byte. Al terminar, apagá los indicadores y descargá el módulo con `rmmod`:

```
$ sudo rmmod tp6_leds
```

Para solicitar los puntos, entregá una explicación breve con capturas de la compilación, la carga del módulo y las pruebas del dispositivo. Prepará una demostración de la implementación.

11. RAID en software (9/10)

TRACK – 9/10

Descripción: crear un arreglo RAID (Redundant Array of Independent Disks) de nivel 1 con `mdadm`, comprobar su estado degradado y reconstruirlo.

Creá y conectá dos discos virtuales vacíos para este track. Identificalos con `lsblk` e investigá las opciones de `mdadm` que vas a usar.

Creá con los dos discos un arreglo RAID1. Creale un sistema de archivos ext4, montalo y escribí un archivo de prueba. Consultá el estado del arreglo con `/proc/mdstat` y `mdadm -detail`; usá `mdadm -examine` para inspeccionar los metadatos de cada disco.

Marcá uno de los miembros como fallado con `mdadm` y retiralo del arreglo. Consultá el estado degradado y comprobá que el archivo de prueba siga accesible. Volvé a incorporar el mismo disco con `mdadm` y observá la reconstrucción del arreglo.

Al terminar, desmontá el sistema de archivos y detené el arreglo.